1. Sign up for free cloud at: https://www.oracle.com/cloud/free/
   Account Name: jdshipp
   Username: jdshipp@loskguild.com
   Pass: XXXXXXXXX (Don't use this password)
2. This is supposed to be “Free”, but in my experience, I had to upgrade to “Pay as you go” to have the ability to create “Compute” instances. As long as what you create is in the “Free Tier", you should not be charged.I made mine in the Chicago datacenter, and I was unable to create Compute instance without first upgrading, This document is a work in progress. Please feel free to make changes. Make sure to choose a US data center.
3. Go Through steps of setting up payment method (supposedly it will always be free though) and setting up mobile authenticator, and eventually get to “Oracle Cloud” dashboard looking screen.
4. Upgrade from Free Tier to Pay as you go:
   a. Hamburger menu at top left, Billing & Cost Management, Upgrade and Manage Payment
   b. Add Payment Method
   c. Upgrade your account, it may take a while to process
   d. Supposedly, if you only use “Free Tier” stuff, you wont get charged..
   e. I tried to create a Compute resource (see below) before upgrading, and it said, “no resources available”... after some reading, this depends on the region you selected and how busy it is… and you can’t change your region once selected.. Someone said that recreated another account and made their region Montreal instead of a US region and it they were able to create the compute resource.
5. Create Storage:
   a. Click hamburger menu at top left and click “Storage”
   b. Click “Block Volume”, “Create Block Volume”, name it whatever: BlockVolume1
   c. Click “Custom” for Volume size and performance, Change volume size to 100gb

      *******After some reading, an option you have is making a 100gb block volume, this keeps you in the “Free Tier” and it will be free, however, some people have reported on reddit that Oracle will delete “Free Tier” accounts randomly (according to the Internet,who knows?!), so they would make their Block Volume 155gb, which barely puts you over the “Free Tier” limit (200gb total, the boot drive for the compute instance is 50gb, and this would be 155gb, putting the total at 205gb), and you would be charges a few cents a month, and this keeps Oracle from deleting your account (also, you need to upgrade from “Free Tier” to “Pay as you Go”, refer to step 3 above). I would think you could stay below the 200gb limit and keep it free and it wouldnt get deleted… not everything you read on the Inernet is true!! And the feeling I get is that this free tier get abused******* I did this and got charged 98 cents the next month.

   d. For “Target volume performance”, slide all the way to right (120)

    e. Select “none” for Backup policies. You can select something beside none, but it will cost a couple dollars per month… i selected none, but I’ll do my own backups manually for now
    f. Click “Create Block Volume”
6. Create Compute (VM):
    a. Click hamburger menu at top left and click “Compute”, “Instances”
    b. Click “Create instance”
    c. Edit “Image and shape”Canonical Ubuntu 24.04 Minimal aarch64”
    d. Click “Change image”, select “
    e. Click “Change Shape”, Check: Ampere, VM.Standard.A1.Flex
    f. Change OCPU to 4 and memory to 24, click “Select Shape” at bottom left
    g. Click “Save private key”, file will download and we will use later and keep this file
    h. Under “Boot Volume”, uncheck “Use in-transit encryption”
    i. Click “Attach block volume”
    j. select volume created in step 4
    k. For attachment type, select custom and paravirtualized, uncheck Use in-transite encryption, select top option for device path (/dev/oracleoci/oraclevdb)
    l. Click “Attach”
    m. Click “Create”
    n. Wait for it to provision - may take a few minutes
7. Change performance of Boot Volume on the Compute instance you just made
    a. Hamburger menu, Compute, Instances, instance you just made
    b. Bottom left menu, click Boot Volume
    c. Click boot volume name
    d. Click edit
    e. For “Target volume performance”, slide scale all the way right (120)
8. Setup domain
    a. You will need a domain for the website to work.
    b. Get the public IP address of the new computer instance we just made: Top left hamburger menu, compute, instances, click your running instance.
    c. Take note of the “Public IPv4 address”. Mine is 64.181.215.42. Yours will be different.
    d. If you already have a domain from somewhere, you can skip ahead to about j
    ~~e. There are some free domain (sub domain) options: https://www.noip.com/remote-access~~
    f. If you don’t have a domain, I recommend cloudflare. Otherwise transfer your domain to cloud flare. Cloudflare is cheapest and best. Go to https://www.cloudflare.com/ and sign up.
    g. Sign in, and go to Domain Registration on left menu, and go to register domains
    h. Search and find a domain you like.. Pay for it. I usually register for the max years I can
    i. Once registered, go to Manage Domains on left menu
    j. Click manage on the right for your domain
    k. Click “Update DNS Configuration” on right

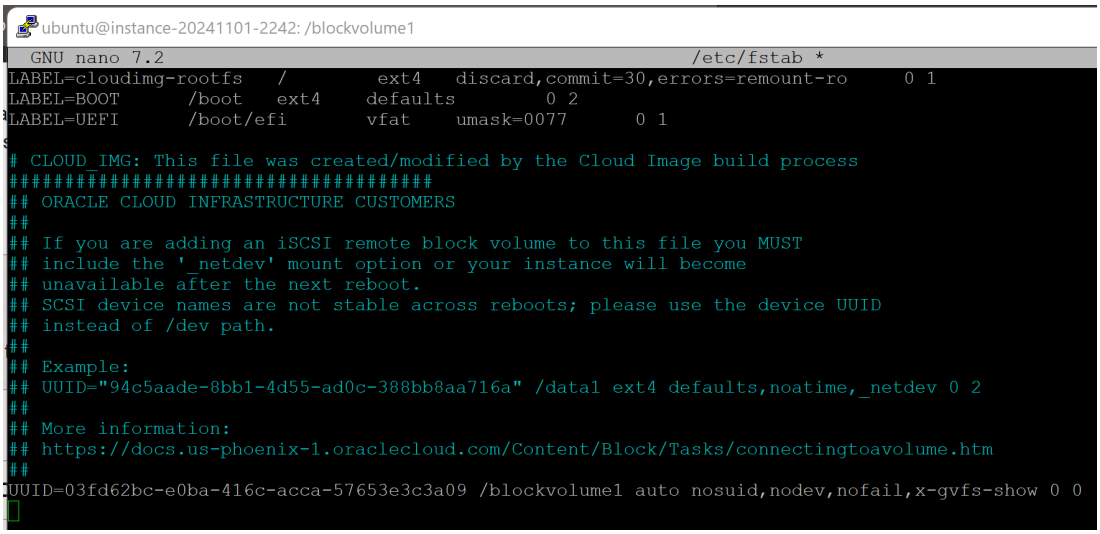
l. Add an “A” record, with “@” for the name, and whatever your public IP address is for the IP address. You do not want to enable “Proxy”, you want “DNS Only”. Click save
m. Add another record, “CNAME” record, for the name enter “www” and for the name enter your domain without www in front, uncheck Proxy, save.
n. Open command prompt on your local PC and ping your domain. It should resolve to your public ip address.

~~9. Setup Domain (free option)~~

~~a. Go to: https://www.noip.com/ and sign up~~
~~b. This account will be for personal, remote access.~~
~~c. Create a hostname. This service is for a free “sub domain” and you have to pick one of the available, free main domains.. I went with “zapto.org” and “isys” for the hostname, which will be “isys.zapto.org”~~
~~d. Record type is “DNS Host (A)” and the public ip address of your compute instance (refer to steps 7b and c)~~
e. ~~You have to manually log into this noip.com and verify you are still using your domain for it to continue to work~~

10. Connect with Putty / SSH:
    a. From home computer, download putty.exe and puttygen.exe from: https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html
    b. Open puttygen.exe
    c. Click “Load”
    d. Select the “.key” file you saved in step 5g
    e. Click “Save private key”
    f. Ignore warning, save the file to whatever filename you want
    g. You should have two files, the original .key file you downloaded when you made the instance, and this .ppk file you just made with puttygen.. Be sure to save these files somewhere safe, or email them to yourself or something, otherwise, you wont be able to connect with putty and there is no way to get the private key
    h. Close puttygen
    i. Open putty.exe
    j. Get the public ip address of the now running instance you made in step 5 and paste it into “Hostname or Ip address” in putty.exe
    k. On the left menu of putty, Connection > SSH > Auth > Credentials, browse to the private key you just made with puttygen.exe and select it
    l. Back to the top option on the left menu in putty.exe “Session”, enter a name for the session and click save… Then you should be able to open putty, open the saved session, and connect to the instance
    m. Click “open” in putty.exe
    n. Enter ubuntu for the username
11. Setup Storage
    a. sudo fdisk -l (shows drives / partitions)
    b. Look for the 100gb drive in my case, it was /dev/sdb, so we need to make a partition on that drive

c. sudo fdisk /dev/sdb
d. p (displays partitions)
e. n (makes new partition)
f. Enter to accept defaults for new partition
g. When new partition is made, type: w (writes partition table)
h. Now we are back to normal prompt, and we need to format the partition
i. sudo mkfs.ext4 /dev/sdb1
j. Next is to mount the disk / partition
k. Make a directory for mount point, type:
   sudo mkdir /blockvolume1
l. Type: sudo blkid
   Get the UUID for sdb1
m. Update package list:
   sudo apt-get update
n. Install text editor nano with the following command:
   sudo apt-get install nano
o. Nano will be installed.. I prefer nano, you may prefer vi
p. Type: sudo nano /etc/fstab
q. Add a line like this to end of file (UUID from previous step):
   UUID=bed82fdf-1597-47c5-a199-50eb07bac6a5 /blockvolume1 auto nosuid,nodev,nofail,x-gvfs-show 0 0



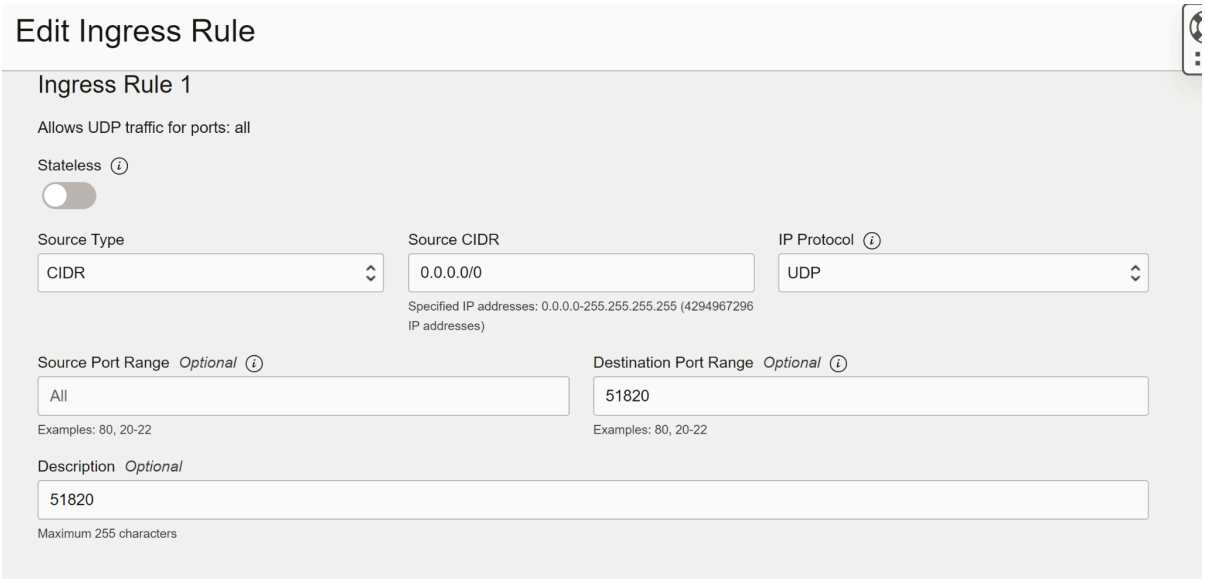
r. Ctrl - o (saves file)
s. Ctrl - x (exits nano)
t. sudo reboot
u. Login and type: mount (or if too busy, type: mount | grep blockvolume1)
v. You should see something mounted on /blockvolume1
w. In my case, sda and sdb got flip flopped after reboot… and that's ok, because /blockvolume1 is mounted to our 100gb drive
x. You can also type "df" to see disk free space.. And verify blockvolume1 mount

~~12. Install Wireguard VPN server on new ubuntu instance~~

a. ~~Once you have successfully connected to the server using putty, enter the following command at the command prompt:~~
   ~~sudo curl -L https://install.pivpn.io | bash~~
b. ~~Accept defaults for the most part, but select ubuntu as the user, and Wireguard as the VPN, 51820 as the port, Quad 9 for DNS, Use Public IP, Yes to upgrades, Yes to reboot~~
c. ~~Log back in with Putty~~
d. ~~These commands manage the VPN: pivpn add, pivpn help~~
e. ~~Type pivpn add~~
f. ~~Enter some name: Justin-Home~~
g. ~~After you hit enter, a client config will be created~~
h. ~~cd /home/ubuntu/configs~~
i. ~~nano Justin-Home.conf~~
j. ~~This opens the client vpn config file created earlier~~
k. ~~Highlight the text of the file and paste into notepad on your local computer.~~
l. ~~Change:~~
   ~~AllowedIPs = 0.0.0.0/0, ::0/0~~
   ~~To~~
   ~~AllowedIPs = 10.0.0.0/24~~
m. ~~If you left it 0.0.0.0/0, then all traffic would be routed through the VPN tunnel.. Changing it to 10.0.0.0/24, then only traffic bound for this network on oracle cloud is routed through the tunnel~~
n. ~~Ctrl-x exits nano~~
o. ~~Save the file on your local computer, make sure the file extension is .conf and not .txt~~

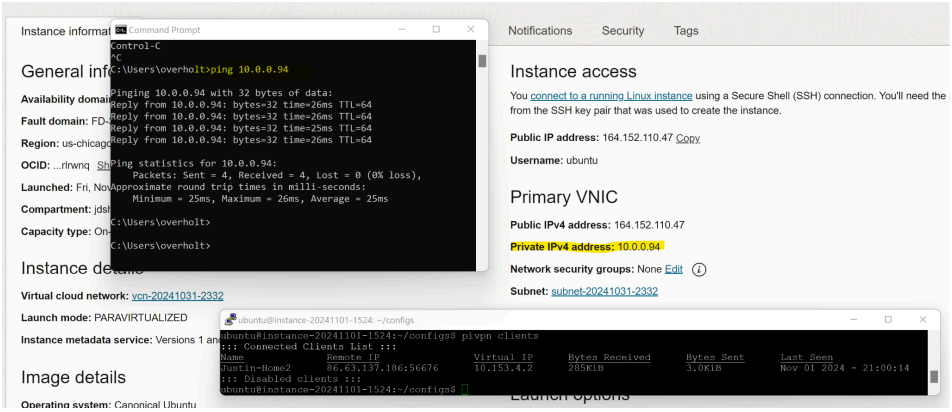
13. Open firewall port on Oracle so the VPN works (AND HTTP ports)
    a. Hamburger menu at top left, Compute, instances, click your running instance
    b. Under “Instance Details”, click your Virtual cloud network
    c. On bottom left menu, click “Security Lists”
    d. Click the default security list

e. Add Ingress rule like the following:



f. Save changes
g. Make another rule, 0.0.0.0/0 for source CIDR, TCP for IP Protocol, 80 for destination port. TCP 80 for description, add ingress rules
h. Add another rule, like previous, but for port 443
i. Back to putty, type the following commands to open firewall port on VM:
   sudo apt-get install ufw
   sudo ufw allow 51820/udp
   sudo ufw allow 80/tcp
   sudo ufw allow 443/tcp

14. Install VPN client on your PC at home/work (you will need to make separate clients for each device you want to connect from.. No need to make the all the clients you want to use from all your devices, like home pc, work pc, laptop, etc.. but you need make one for the computer you are working from now) Later, you can start at step 7e to make more clients for other devices
    a. Download Wireguard Client for windows from: https://www.wireguard.com/install/
    b. Import tunnels from file, browse to the file you just made and open it
    c. Click activate
    d. Check the VPN is working by ping the private ip address of the new server on Oracle cloud (Instances, running instance, Primary VNIC, Private IPv4 address) from a command prompt from your local PC. You can also check the VPN connection through the putty connection with command: pivpn clients



Take note of the private IP address of the server….

15. Install Docker on Ubuntu
    a. Docker is virtualization tech that allows quick easy deployment of a Docker "Container"… which is a simple app (like a web app) or a full blown server (like SQL server). that can be run on any computer running Docker.
    b. Type the following commands in putty:
        i. sudo apt update
        ii. sudo apt upgrade
        iii. sudo systemctl disable snapd.service
        iv. sudo systemctl disable multipathd.service
        v. sudo snap remove core
        vi. sudo apt purge snapd
        vii. sudo apt install unattended-upgrades
        viii. sudo dpkg-reconfigure --priority=low unattended-upgrades
        ix. sudo apt install apt-transport-https ca-certificates curl software-properties-common
        x. curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
        xi. echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
        xii. sudo apt update
        xiii. apt-cache policy docker-ce
        xiv. sudo apt install docker-ce
        xv. sudo systemctl status docker

c. Verify that Docker is active and running with the previous command

```
ubuntu@instance-20241101-1524:~/configs$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2024-11-01 21:27:42 UTC; 34s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 2608 (dockerd)
      Tasks: 10
     Memory: 20.8M
        CPU: 454ms
     CGroup: /system.slice/docker.service
             └─2608 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Nov 01 21:27:42 instance-20241101-1524 dockerd[2608]: time="2024-11-01T21:27:42.054744938Z" lev
Nov 01 21:27:42 instance-20241101-1524 dockerd[2608]: time="2024-11-01T21:27:42.056667186Z" lev
```

d. To see info on Docker, type: sudo docker info
e. To test if the whole docker system is working, type: sudo docker run hello-world
f. To Show running Docker containers: sudo docker ps
g. To Show all Docker containers: sudo docker ps -a
h. To remove a container: sudo docker rm <name or id>
i. sudo reboot (next step of installing SQL Server gave trouble, unless we reboot)
j. sudo docker logs -f --tail 100 sqlserver

16. Install SQL Server Docker Container
    a. We need to make a random password for use with the SQL Server “sa” account (built in admin account). It has to be complex enough, or the installation will fail. Browse to: https://passwordsgenerator.net/ set password length to 32, for “Include Symbols”, set it to use only: !#$
    b. Click generate, pick and password and take note of it.
    c. This is the password I will use for this example (be sure you use something else): k9dQPLx$TW3AKsuR#DUz6f!48FapEtqX
    d. Back to putty, type cd /blockvolume1
    e. Type “ls”, directory should be empty (I had lost and found or something, but thats ok)
    f. ~~Orange PI zero 2w: dgMZr2fXZuungxK7XmbAEHEMcsg1e3Fn~~

       ~~sudo docker run -it -e 'ACCEPT_EULA=1' -e 'MSSQL_SA_PASSWORD=dgMZr2fXZuungxK7XmbAEHEMcsg1e3Fn' -p 1433:1433 -v /var/sqldir:/var/opt/mssql --user root --privileged=true --name sqlserver mcr.microsoft.com/azure-sql-edge:1.0.7~~

    g. Type this (substitute you password for the one I use, and the directory names if you used something different):
    h. sudo docker run -it -e 'ACCEPT_EULA=1' -e 'MSSQL_SA_PASSWORD=k9dQPLx$TW3AKsuR#DUz6f!48FapEtqX' -p 1433:1433 -v /blockvolume1/sqldir:/var/opt/mssql --user root --privileged=true --name sqlserver mcr.microsoft.com/azure-sql-edge:1.0.7
    i. This could take a while (several minutes) ** This worked for me, first shot, on 4/23/25 (skip to n.)

j. It will probably fail (if you see, near the end, something about “Not compressing the dump files”, then it failed. I cannot figure out why it fails… and I spent a lot of time trying to figure it out.. If you figure it out, please let me know.
k. So, if it failed, you need to remove the failed container and the directory it just made in our /blockvolume1 directory.. Type “ls” to see if there is a “sqldir” directory.. There probably is, and we need to delete it and remove the failed container… Type the following two commands, then start over at step 13g to try and run it again… You may need to go through this sequence of running the 13g command to create the docker container and if it fails, running the next to commands again… took me 4 times for it to finally work.. It bugs me that it fails like this, then works… From what I can gather, it something about a newly released linux kernel that causes this… You can tell when it works… Ok the two commands to run after it fails:
l. sudo docker rm sqlserver
m. sudo rm -R -f /blockvolume1/sqldir
n. ******* Press up arrow on keyboard to repeat commands *********
o. Ok, once you get it working, CTRL-C to stop SQL Server
p. Type the following:
sudo docker start sqlserver && sudo docker update --restart unless-stopped sqlserver
q. Now SQL Server is installed, running, and will start automatically after reboot

17. Connect to SQL Server with SQL Server Management Studio
    a. Verify your wireguard connection is up and you can ping the ip address of the server (you may have just completed this with step 11d)
    b. You need to download and install the latest SQL Server Management Studio:
    https://learn.microsoft.com/en-us/sql/ssms/
    MAC users can use Azure Data Studio:
    https://learn.microsoft.com/en-us/azure-data-studio/
    But the interface is very different, but it’s the only option for MAC users
    c. Once downloaded and installed, open SQL Server Management Studio… and connect to the server. Use “sa” for the username, and the password you generated and used to install for the password, check remember password, select optional for encryption, and check trust server certificate.
    d. Click connect (this is awesome!)
18. Create Simple Database for testing
    a. New Query in SQL Server Management Studio (make sure you have the new sql server selected in object explorer)
    b. Paste in the following: check the comment
    c. This is different from that databases we make for students in that the users are created at the database level for the students (called “partial containment”), which I like, but this version of SQL Server (Cloud version of SQL called Azure Edge SQL, but based on same engine as full blown SQL Server) doesn’t support partial containment, so we have to create the users at the server level, which is a little different and more complicated.

d. Create table in new database: check comment

19. Create Web Application and test database connection
    a. Open Visual Studio 2022 (may want to update to latest version)
    b. Create New project
    c. Select “ASP.NET Core Empty”
    d. Name it “Geo”
    e. Select latest framework (.net 9 comes out in a couple of weeks, in mid november)
    f. Check: Configure for https
    g. Uncheck: enable docker (we could do this, and maybe best practice is to use docker and container for isolation / security, but it’s more complicated)
    h. Check: do not use top level statements
    i. Once loaded, click “Project” menu and Manage Nuget Packages, browse / search for Microsoft.Data.SqlClient and install it. Close Nuget management tab.
    j. Right click project name (Geo) in solution exploder, add, class, name it “Walton_DB.cs”.
    k. Paste in the following, replacing the IP address and password to whatever yours is for the connection string:
       https://pastebin.com/ejRc1aXx
       Password: r2scZagP9q

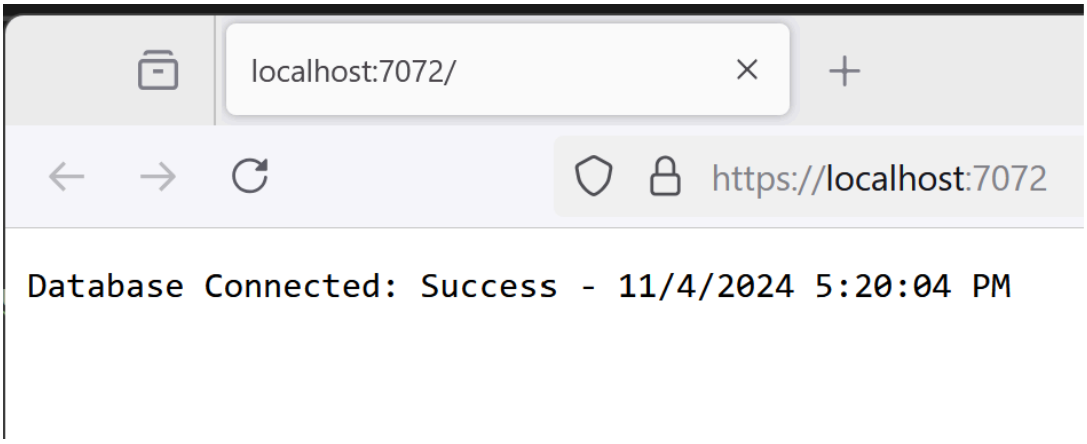
    l. Save Walton_DB.cs
    m. Back to program.cs
    n. Replace at about line 10:

```
app.MapGet("/", () => "Hello World!");
```

With

```
string LoginResult, SQLServerTime;
if (Walton_DB.OpenConnection())
{
   LoginResult = "Success";
   // SQL Server will show GMT Time
   SQLServerTime = Walton_DB.RetrieveSingleValue("SELECT
GETDATE()").ToString();
}
else
{
   LoginResult = "Failed";
   SQLServerTime = "Unknown";
}
app.MapGet("/", () => $"Database Connected: {LoginResult} -
{SQLServerTime}");
```

o. Run the application. It will launch a web browser and look like this:



p. If you get Failed and unknown, then there is a problem, which could be:
   i. Your VPN (Wireguard) connection is down, try and reconnect and ping the IP address of the server, like in previous steps
   ii. SQL Server is down and not running. You can check the logs with this command: sudo docker logs -f --tail 100 sqlserver
   iii. You should be able to figure out if its running or not based on the output of the logs
   iv. You can also do: sudo docker ps
   v. If you add "-a" to the above command, it will show non running docker containers
   vi. Maybe your connection string is wrong. It has to have the correct IP address, user login, and password

q. Once your database connection and database is working, we will go to the next step

20. Continue with web app
   a. Comment out the code we used to test the database
   b. Add this between the var builder and var app lines (about lines 7 & 8):
      builder.Services.AddRazorPages();
   c. Add this below the line "var app = builder.Build();" (about line 10 now):

```
if (!app.Environment.IsDevelopment())
{
   app.UseExceptionHandler("/Error");
   app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();
```

   d. Should look like this:

e.
```
public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);
    builder.Services.AddRazorPages();
    var app = builder.Build();

    if (!app.Environment.IsDevelopment())
    {
        app.UseExceptionHandler("/Error");
        app.UseHsts();
    }
    app.UseHttpsRedirection();
    app.UseStaticFiles();
    app.UseRouting();
    app.MapRazorPages();

    //app.MapGet("/", () => "Hello World!");

    //string LoginResult, SQLServerTime;
    //if (Walton_DB.OpenConnection())
    //{
    //    LoginResult = "Success";
    //    // SQL Server will show GMT Time
    //    SQLServerTime = Walton_DB.RetrieveSingleValue(
    //}
    //else
    //{
    //    LoginResult = "Failed";
    //    SQLServerTime = "Unknown";
    //}
    //app.MapGet("/", () => $"Database Connected: {Logir

    app.Run();
}
```

f. Right Click project name (Geo) in solution explorer, add, New Folder.. Name folder “Pages”
g. Right click pages folder, add, Razor Page, Razor Page - Empty, Click Add, the name “Index.cshtml” is ok, click Add
h. This made two files, One Index.cshtml, which will have html in it, and a “code behind” file, Index.cshtml.cs, which is where we will put any method we need
i. Double click Index.cshtml. Should look like this:

```
@page
@model Geo.Pages.IndexModel
@{
}
```

j. Paste the following below the text above:
https://pastebin.com/Cx0p0YCm
Password: TKeTNZuqCj
k. Save
l. Right click pages folder in solution explorer, add, new item, find “Razor View Import”, set the name to “_ViewImports1.cshtml”, click add
m. Open the file and paste in:
@using Microsoft.AspNetCore.Mvc.RazorPages

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

n. Save file and close tab
o. We will come back to this

21. Add Walton_ChatGPT.cs (Skip this if you don't care to use ChatGPT)
    a. Right click Project in solution explorer, click add, click class, name it Walton_ChatGPT.cs
    b. Copy the following into the BODY of the Walton_ChatGPT class (in between the opening and closing curly braces):
       https://pastebin.com/5iaUwn5L
       Password: M34hS3mQJQ
    c. You will need an API key from platform.openai.com
22. Need to add Walton_Download.cs
    a. Right click Project in solution explorer, click add, click class, name it Walton_Download.cs
    b. Copy the following into the BODY of the Walton_Download class (in between the opening and closing curly braces):
       https://pastebin.com/2h9gFvje
       Password: Lvi2wVMD9w
    c. Save and Close Tab
23. We need to add a place to store global variables and misc methods… This is special, and really cool, because each session/client that browses this website has access to the same variables This works REALLY well for caching data and minimizing database hits.
    a. Right click Project in solution explorer, click add, click class, name it Walton_Globals.cs
    b. Add this one method to the body:
       https://pastebin.com/vch1bbxu
       Password: Dr3TuXdXPe

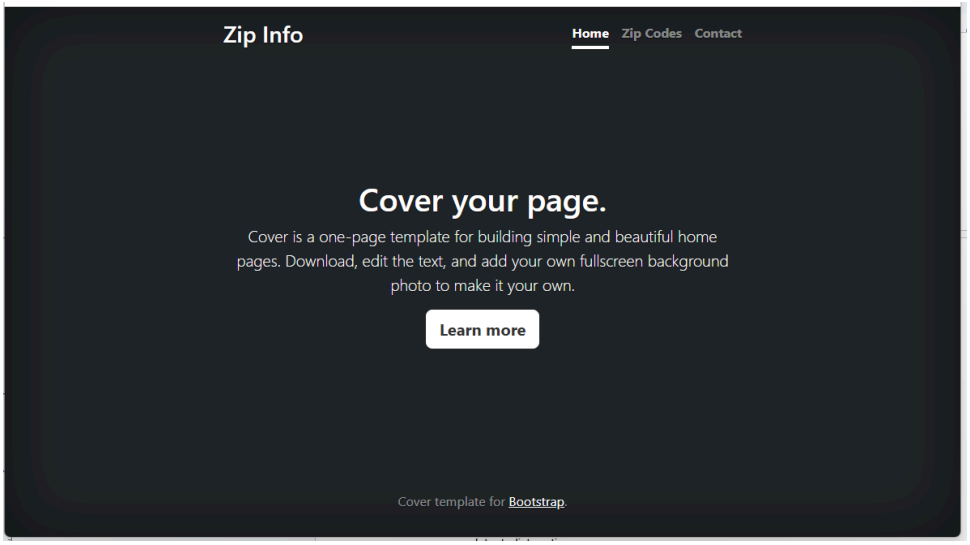
    c. We could put all of these into one big class, but it makes sense to keep them separate, in case you don't need some functionality of a class, you can just not include it in the project.
    d. These classes that we added are static, meaning that only a single instance of the class will be made and shared access sessions… Otherwise, each user that browses to the website would create a new instance of the class.. And we don't want that… it's very resource intensive and the objects we are using are thread-safe (httpclient and sqlconnection) and can be used by multiple threads at the same time - COOL STUFF!
24. Setup SSL Certs
    a. Install LettuceEncrypt, by Nate McMaster, nuget package into project
    b. Paste the following below about like 10, under "builder.Services.AddRazorPages();":
       https://pastebin.com/R0nuDpfb
    c. Password: 5qt3bQdkZb

d. Generate a new password and put it where is says "PutYourOwnPasswordHere…"
e. Open appsettings.json, and paste the following below the first opening curly bracket and above "Logging": https://pastebin.com/f0RgRKdF
f. Password: vJdWRRbpgD
g. Replace the two domains with your domain and your domain with www in the front
h. Enter your email address
i. Save

~~25. Install dotnet runtime on server~~

~~a. wget https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/packages-microsoft-prod.deb -O packages-microsoft-prod.deb~~
~~b. sudo dpkg -i packages-microsoft-prod.deb~~
~~c. sudo apt-get update~~
~~d. sudo apt-get install -y dotnet-runtime-8.0~~
~~e. dotnet --list-runtimes~~

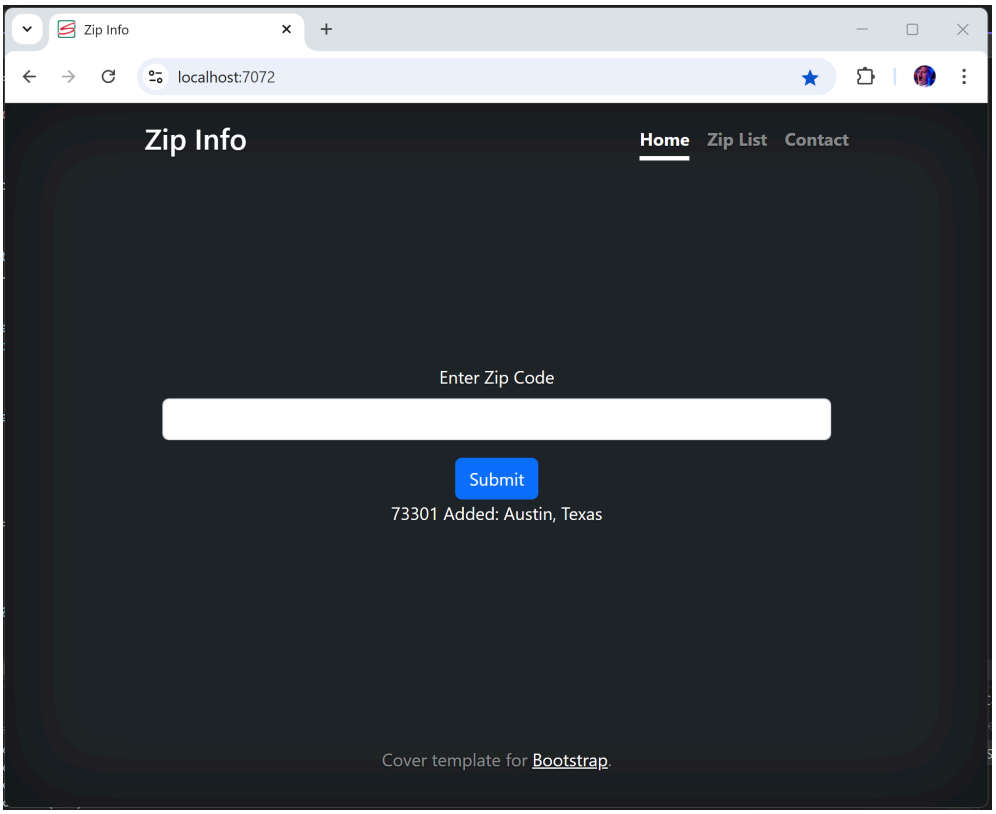
26. Back to the application, Add Zip Code
    a. Run the app and make sure you get a screen like this:



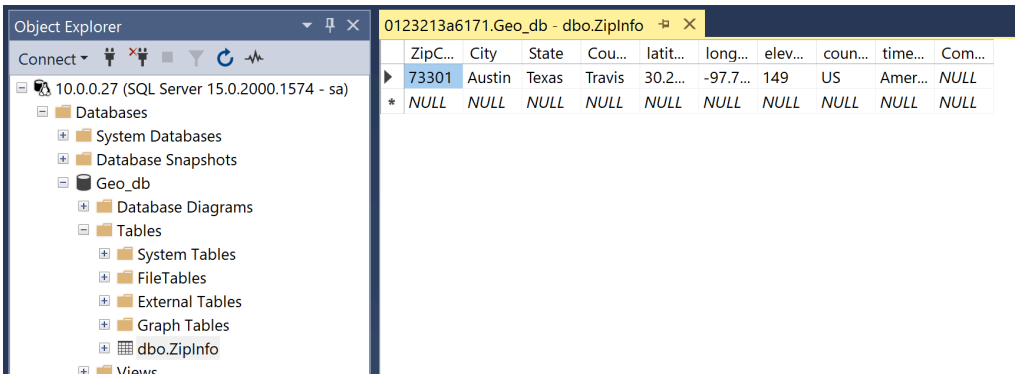
    b. Back to Index.cshtml, at about line 113 is "<main class="px-3">". Delete everything between that and the closing </main> tag. Do not delete the opening and closing <main> tags though.
    c. Paste the following between the <main> tags: https://pastebin.com/ezT0bu4B
       Password: eTktF36QN3
    d. This added a form with a textbox (<input> tag) for zipcode and a button. Then the user enters a value in the text box, and clicks the button, the code is run in Index.cshtml.cs in the "OnPost" event.
    e. Open "Index.html.cs"

f. Add the following under the opening curly bracket for the class:
   [BindProperty]
   public string UserInput { get; set; }
g. This allows us to capture the value of the textbox/input when submitted
h. We need to add two more fields to our table in our database. Run the following in a new Query window in SQL Server management studio:
   ALTER TABLE dbo.ZipInfo ADD
   DateTimeZipAdded2DB datetime NULL,
   DateTimeChatGPTCommentAdded2DB datetime NULL
i. Add the following two methods to to the body of the class, under the "OnGet" method that should already be there:
   https://pastebin.com/6FaNhaEh
   Password: ZnF8KL114b
j. These two methods use both the Walton_DB and Walton_Download helper classes we added earlier
k. Next, we add the following method below the OnGet() Method in the Index.cshtml.cs file (same file we have been working in)
   *** May need to comment out "Walton_Globals.RefreshTableRows();" with // to get rid of error until later:
   https://pastebin.com/5SM58WnL
   Password: ybdBA6gLfW
   This OnPost() method is an event method, and fires when a form button is clicked (in this case, then someone clicks the Submit button for entering a zip code.

l. At this point, you should be able to run the application, type a zip code and click submit. You should get some message back if it failed or was successful.



m. Go into sql server management studio, right click the table, edit, and verify the zip code you just entered is in the database.



27. Application - ChatGPT Commentary

a. In Index.cshtml.cs, add the following method below the ZipInDB method: https://pastebin.com/vux4TH54
Password: sTkeXjXaMh

b. This method is different than other methods we have used. Our other methods start with “private void”, but this one starts with “private async Task”. This means if we call it as a Task, it will create a new thread and run the method separate from the thread we are on. And more importantly, it WILL NOT BLOCK THE CALLING METHOD, which is OnPost(). Because we know the chatGPT api can

sometime be slow, we do not want the users of our webpage to have a bad experience by waiting for the website to respond after they click submit, so we call the slower / chatgpt part of this code asynchronously so the “OnPost()” method still executes quickly and our users are not waiting for the web page to respond.

c. Also, since we added the two new fields to the table “DateTimeZipAdded2DB” and “DateTimeChatGPTCommentAdded2DB”, and our code is setting the value of the fields at the appropriate times, we can run a query later to see just how long chatgpt takes to process our prompts.

d. Go to the OnPost() method paste the following below the line “Message2User = $"{UserInput} Added: {CityState}":

```
// Fire-and-forget call to an async chatGPT method
Task.Run(async () =>
{
   await GetAndSetCommentFromChatGPT(UserInput, CityState);
});
```

e. It should look like this:

```
44          if (AddZip2DB(UserInput, ref CityState))
45          {
46              Message2User = $"{UserInput} Added: {CityState}";
47              // Fire-and-forget call to an async chatGPT method
48              Task.Run(async () =>
49              {
50                  await GetAndSetCommentFromChatGPT(UserInput, CityState);
51              });
```

f. Run the application and add a zip code, then check the database. It should have a new record with commentary and two time stamps:

ZipC...	City	State	Cou...	latit...	long...	elev...	coun...	time...	Commentary	DateTimeZipAdded2DB	DateTimeChatGPTCommentAdded...
71854	Texarka...	Arka...	Miller	33.4...	-94.0...	110	US	Amer...	Texarkana, Arkansas, is unique for being ...	2024-12-14 16:36:32.637	2024-12-14 16:36:38.257
73301	Austin	Texas	Travis	30.2...	-97.7...	149	US	Amer...	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

We see the comments about Texarkana and the two time stamps… looks like chatgpt took about 6 seconds

28. Application - Display Zip Codes in System

a. Add the following to the Walton_Globals.cs, at the top of the class, above the sendemail method:
https://pastebin.com/PLrjLARF
Password: pajCk3JBwf

b. The variable ZipCodeTableRows_html persists in memory as long as the application is running. And this is accessible by all clients. So, we will take advantage of this:

   i. The zip code data in the database does not change often.
   ii. The first time someone pulls up the website, the variable will be empty, do a call will be made to the database to get all the zip code data. We loop through the data and store it as preformatted table html.
   iii. That table html sits in the variable

iv. When the next person goes to the webpage, our logic see that the variable is not empty, so it does not hit the database, but instead spits out the contents of the variable.

v. This saves calls to the database and is very performant

vi. When zip code data in the database does change.. And we know when it changes, we can just call public method in Walton_Globals "RefreshTableRows". So lets do that.

c. Add the following line to the Index.cshtml.cs file in the OnPost() method, under that code we previous added for the Task.run, etc… about line 53:
*** While working through the next steps of this step, you may have errors about variable that do not exist… that's ok, you will make them in the next few steps***
Walton_Globals.RefreshTableRows();
Should look like this:

```
{
    string CityState = "";
    if (AddZip2DB(UserInput, ref CityState))
    {
        Message2User = $"{UserInput} Added: {CityState}";
        // Fire-and-forget call to an async chatGPT method
        Task.Run(async () =>
        {
            await GetAndSetCommentFromChatGPT(UserInput, CityState);
        });

        Walton_Globals.RefreshTableRows();
    }
    else
    {
```

d. Lets also add the same line to the bottom of the GetAndSetCommentFromChatGPT method. Looks like:

```
private async Task GetAndSetCommentFromChatGPT(string Zi
{
    try
    {
        string ChatGPTResult = Walton_ChatGPT.GetAnswer(

        SqlCommand cmd = new SqlCommand();
        cmd.CommandText = "UPDATE ZipInfo SET Commentary
        cmd.CommandType = System.Data.CommandType.Text;
        cmd.Parameters.Add("@ZipCode", System.Data.SqlDb
        cmd.Parameters.Add("@Commentary", System.Data.Sc
        Walton_DB.ExecSqlCommand(ref cmd);

        Walton_Globals.RefreshTableRows();
    }
    catch (Exception ex)
```

e. Now, when the data in the database does change, it will automatically be refreshed when someone adds a new zip

f. We need to add some javascript to the index.cshtml file. At the bottom, below the <script> tag that adds bootstrap javascript, add the following: https://pastebin.com/y2kZQBvK

```
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384
<script>
    // Initialize all popovers
    document.addEventListener('DOMContentLoaded', function () {
        const popoverTriggerList = [].slice.call(document.querySelectorAll('[data-bs-toggle="popover"]'));
        const popoverList = popoverTriggerList.map(function (popoverTriggerEl) {
            return new bootstrap.Popover(popoverTriggerEl);
        });
    });
</script>
ody>
```

g. In the same file, before the closing </main> tag, add the following: https://pastebin.com/mVYW8jxV

```
            @* Only display the paragraph if UserInput is n
            @if (TempData["MessageToUser"] != null)
            {
                <p>@TempData["MessageToUser"]</p>
            }

        <hr />
        <table class="table table-striped table-dark">
            <thead>
                <tr>
                    <th scope="col">Zip</th>
                    <th scope="col">City</th>
                    <th scope="col">State</th>
                    <th scope="col">Info</th>
                </tr>
            </thead>
            <tbody>
                @Html.Raw(Model.HtmlRows_Local)
            </tbody>
        </table>
    </main>
```

h. Now, back to the index.cshtml.cs file, add the following class level property / variable and add the code to the OnGet() Method: https://pastebin.com/S9xbwHLt

Password: qSWy45tEPY

```
namespace Geo.Pages
{
    5 references
    public class IndexModel : PageModel
    {
        [BindProperty]
        9 references
        public string UserInput { get; set; }
        [BindProperty]
        2 references
        public string HtmlRows_Local { get; set; }

        0 references
        public void OnGet()
        {
            if (Walton_Globals.ZipCodeTableRows_html == "")
            {
                Walton_Globals.RefreshTableRows();
            }
            HtmlRows_Local = Walton_Globals.ZipCodeTableRows_html;
        }

        0 references
        public IActionResult OnPost()
```

i. Run the application, click the info buttons to see comments chatgpt said about the city

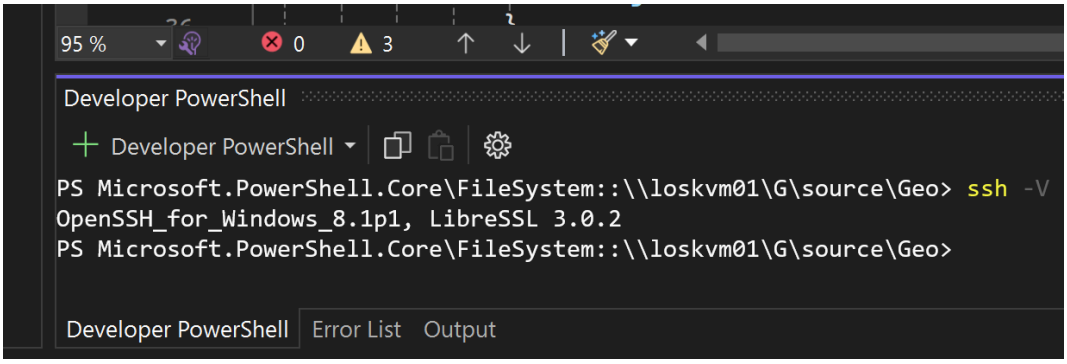
29. This is about it for the application, we will deploy it to the server next. But I have some thoughts:
    a. This is a single page application. To make it multi page, you would need to implement “layouts”, which allows common layouts on all pages of the website.. Ask chatGPT about this if you want to do it. I could also demo it later if needed.
    b. The security of this application is minimal. There are ways to make it more secure. After you get it deployed to the server, you can check basic security with: https://developer.mozilla.org/en-US/observatory I could demo this too
    c. If you want a challenge, display somewhere on the webpage the average, minimum, and maximum time, in seconds, it took ChatGPT to respond to the prompts. Use caching so it does have to hit the database everytime someone visits the website, but update the stats if someone does add a new zip. I can demo this too
30. Fix server firewall and something in Program.cs
    a. When trying to get this to run on the server as a non-root user, I ran into some problems. Any port below 1000 is considered a "privileged port” and non-root users can’t bind to those ports. So, the way we do it is use some firewall rules to redirect ports 80 and 443 to 8080 and 8443. To do this, we need to do the following:
    b. Type the following in putty:
    sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080

And

sudo iptables -t nat -A PREROUTING -p tcp --dport 443 -j REDIRECT --to-port 8443

c. ~~Type this: sudo apt install -y iptables-persistent~~
d. ~~Type this: sudo netfilter-persistent save~~
e. Now we have to change the ports the application listens on
f. Go to program.cs
g. In the gray text, starting about like 12 with #if !DEBUG
h. Find the line with options.ListenAnyIP(80) and change the 80 to 8080
i. Find 443 and change it to 8443

31. Deploying application to server
    a. We are going to do the simplest deployment. If you want to host multiple domain and web pages on the same server, you will have to use a more complicated approach.
    b. We cant deploy in a one-click easy way built into visual studio. We will need to build some commands. Use notepad and not Word or Google docks to build the commands… Word will do weird stuff with single quotes
    c. In visual studio, down around my error list at the bottom, there is a Developer Powershell Tab.
       Run the command: ssh -V
       This verifies openSSH is installed on your computer.



       If it's not installed.. Figure out how to install it.. I have Windows 11 and it was already installed.
    d. Open PuttyGen and open the ppk file you made earlier for connecting with putty. We need to make another type of file that work with openssh. Click Conversion menu, and click Export OpenSSH Key. Name the file id_rsa with no extension
    e. Use the following command to test connection in Visual Studio dev powershell:
       ssh -i C:\path\to\your\private-key user@your-server

       This is what I used (the server can be public or private ip, or domain name.. I think):
       ssh -i C:\Users\overholt\Downloads\id_rsa ubuntu@10.0.0.27

       Answer yes to the security question.. If all is correct, you will be at a console

prompt on the server on Oracle (cool)

Type exit to leave the prompt

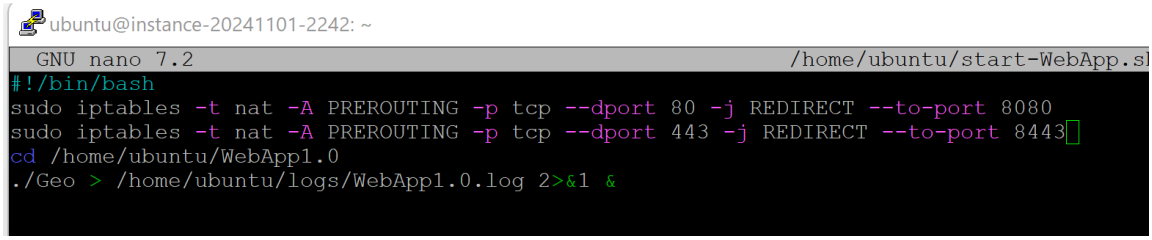
f. Log into putty and type:
   cd <enter>
   mkdir WebApp1.0
g. Compile the application
   Type the following in VS powershell:
   dotnet publish -c Release -r linux-arm64 --self-contained true /p:PublishSingleFile=true
h. Copy files from your computer to server:
   scp -i C:\path\to\your\private-key -r C:\path\to\publish user@your-server:/var/www/myapp

   scp -i C:\Users\overholt\Downloads\id_rsa -r \\loskvm01\G\source\Geo\Geo\bin\Release\net9.0\linux-arm64\publish* ubuntu@10.0.0.27:/home/ubuntu/WebApp1.0

   I used a network path above, you will probably use something like C:\users\johndoe\source\repo\Geo\publush4Oracle or something like that, instead of \\loskvm01\etc\etc
i. After files are copied, we need to go back to putty and go to the Webapp1.0 directory and change the app to executable. Type: chmod +x Geo
j. You can type “ls” and verity the Geo file is green (indicated it’s executable)

32. Run Application on the server
    a. In putty, go to the directory that has the green text Geo/… should be directory WebApp1.0
    b. Type:
       ./Geo (That’s period, slash, Geo)
       Enter
    c. This starts up the application.. This took me a lot of trial and error to get this working without a bunch of errors. I could get it to run as root, but not as non-root… But I believe the instructions work for non-root now
    d. Do not run it with sudo.
    e. You may need to reboot
    f. Once it looks like its running without errors, browse to your domain and it should come up
    g. CTRL-C to stop it
33. Set app to restart with server
    a. mkdir /home/ubuntu/logs
    b. nano /home/ubuntu/start-WebApp.sh
    c. Type the following into the file
       #!/bin/bash
       sudo ufw allow 51820/udp

sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
sudo iptables -t nat -A PREROUTING -p tcp --dport 443 -j REDIRECT --to-port 8443
cd /home/ubuntu/WebApp1.0
./Geo > /home/ubuntu/logs/WebApp1.0.log 2>&1 &



d. CTRL-O to write the file, CTRL-X to exit
e. chmod +x start-WebApp.sh
f. sudo apt install cron -y
g. crontab -e
h. Go to bottom and type: @reboot /home/ubuntu/start-WebApp.sh
i. CTRL-O
j. CTRL-X
k. sudo reboot
l. When it comes back up, you should be able to browse to your domain and it shows the website
m. If you want to deploy a new version of the website, you need to kill the process. Type: ps -A
n. Look for Geo, and get it's process number on left
o. Type kill -9 processnumber
p. This will stop the application and you can scp your new version over.
q. After you have copied new version over with scp, you can restart the application by typing /home/ubuntu/start-WebApp.sh
r. Type ps -A to verify it's running
s. You can also type: tail -f /home/ubuntu/logs/WebApp1.0.log
t. This gives you the logs of the web application
u. Mine should be working now: https://stocksandlocks.com/ (as of 12/17/24 it is not working… I made some firewall rule wrong and locked myself out and had to start all over……)

34. Securing / Hardening
    a. Go to: https://developer.mozilla.org/en-US/observatory
    b. Put in your domain and let it scan
    c. It will grade you and tell you whats wrong
    d. You can Google the error and try and fix it, but ChatGPT is great for this.

e. For example, one of my problems was:
   Cookies set without using the Secure flag or set over HTTP.
f. So, I asked ChatGPT:
   I have an asp.net core razor page app running on .net 9.0 on Kestrel. how can I fix this: Cookies set without using the Secure flag or set over HTTP.
g. It gave me a bunch of possibilities to fix it.
h. algoarmada.com has near perfect score